Tổng quan bài báo TagRAG

Bài báo gốc: Tao et al. (2026) • *Findings of the Association for Computational Linguistics: ACL 2026* (tr. 6434–6456) • **Đơn vị:** East China Normal University (ECNU) • **Mã đề tài:** LM40

> Đồ án CK (30%) + Seminar về ĐACK (10%) + Vấn đáp về ĐACK = 50%
>
> **Cơ cấu đánh giá học phần (50% tổng điểm môn NLP):** Đồ án Cuối kỳ chiếm **30%** (mã nguồn, thực nghiệm & báo cáo khoa học), Seminar thuyết trình chiếm **10%** và Vấn đáp trực tiếp chiếm **10%**. Tài liệu này phân tích toàn diện thuật toán TagRAG và xây dựng lộ trình thực thi cho cả 3 hạng mục.

1. Tổng quan Bài báo & Động lực Nghiên cứu (Motivation)

Kỹ thuật Sinh văn bản tăng cường truy xuất (Retrieval-Augmented Generation – RAG) do **Lewis et al. (2020)** khởi xướng đã trở thành nền tảng cốt lõi giúp giảm thiểu ảo giác cho các Mô hình Ngôn ngữ Lớn (LLMs) trong các miền chuyên sâu như y tế, pháp luật và khoa học máy tính. Tuy nhiên, phương pháp RAG truyền thống (*NaiveRAG*) chỉ truy xuất các đoạn văn bản rời rạc (fragment-level chunks), dẫn đến thất bại trước các câu hỏi đòi hỏi **tóm tắt hướng truy vấn trên toàn bộ kho dữ liệu (Query-Focused Summarization – QFS)** hoặc suy luận tổng thể.

Sự ra đời của **GraphRAG** (**Edge et al., 2024**) cùng các biến thể phân cấp như **RAPTOR** (**Sarthi et al., 2024**) và **HiRAG** (**Huang et al., 2025a**) đã giải quyết bài toán toàn cục bằng cách trích xuất đồ thị thực thể, phân hoạch cộng đồng (thuật toán Leiden) và sinh bản tóm tắt cho từng cộng đồng. Dù vậy, các kiến trúc GraphRAG hiện hành vấp phải hai rào cản lớn khi triển khai thực tế mà **Tao et al. (2026)** đặt làm câu hỏi nghiên cứu trung tâm:

- **(RQ1) Làm thế nào để biểu diễn và suy luận toàn cục trên đồ thị tri thức với chi phí tài nguyên thấp?** Các hệ thống như GraphRAG phụ thuộc nặng nề vào các LLM khổng lồ (như GPT-4o) ở mọi khâu trích xuất, phân cụm và sinh câu trả lời. Khi chuyển sang các mô hình ngôn ngữ nhỏ (Small LLMs từ 1.5B đến 4B tham số), chất lượng suy luận sụp đổ hoàn toàn. Đồng thời, các phương pháp tối giản cấu trúc như **LightRAG** (**Guo et al., 2025**), **KET-RAG** (**Huang et al., 2025b**), **MiniRAG** (**Fan et al., 2025**), **FG-RAG** (**Hong et al., 2025**), **PathRAG** (**Chen et al., 2025a**) hay **LeanRAG** (**Zhang et al., 2026**) chỉ tập trung lấy đồ thị con cục bộ, làm mất đi bức tranh tổng quan toàn kho dữ liệu.
- **(RQ2) Làm thế nào để cập nhật tăng trưởng tri thức (Incremental Knowledge Insertion) hiệu quả?** Quy trình xây dựng từ dưới lên (bottom-up) của GraphRAG đòi hỏi phải phân chia lại cộng đồng và tóm tắt lại toàn bộ đồ thị mỗi khi có tài liệu mới thêm vào, gây lãng phí thời gian và chi phí tính toán khổng lồ.

> **Điểm đột phá cốt lõi của TagRAG (Tao et al., 2026)**
>
> Thay vì gom cụm cộng đồng mù quáng từ dưới lên, **TagRAG** tổ chức tri thức theo kiến trúc hai tầng hướng dẫn bởi nhãn: liên kết các **Nhãn Thực thể (Object Tags)** cấp thấp với một **Nhãn Miền gốc (Root Domain Tag)** định trước để tạo thành các **Chuỗi Nhãn Miền phân cấp (Hierarchical Domain Tag Chains)** dưới dạng Đồ thị có hướng không chu trình (DAG). Thiết kế này giúp TagRAG đạt tỷ lệ thắng trung bình **78.36%** trước các đối thủ trên bộ chuẩn UltraDomain, đồng thời **nhanh hơn 14.6 lần khi xây dựng đồ thị** và **nhanh hơn 1.9 lần khi truy xuất** so với GraphRAG ngay trên mô hình nhỏ Qwen3-4B.

2. Định nghĩa Toán học & Kiến trúc Hai Giai đoạn của TagRAG

Cho tập văn bản đầu vào thuộc một miền chuyên môn $D = \{d_i\}_{i=1}^{|D|}$ và một nhãn miền gốc định trước $\hat{v}$ (ví dụ: *Computer Science*, *Legal*, *Agriculture*). Mục tiêu của TagRAG là xây dựng một **Đồ thị Tri thức Nhãn Phân cấp (Hierarchical Tag Knowledge Graph)** $\mathcal{G}$ kết nối hai hệ thống đồ thị con:

- **Đồ thị Nhãn Thực thể (Object Tag Knowledge Graph):** $\mathcal{G}_o = (\mathcal{V}_o, \mathcal{E}_o)$, trong đó $\mathcal{V}_o$ là tập các từ khóa chuyên ngành cấp thấp được trích xuất trực tiếp từ văn bản và $\mathcal{E}_o$ là các quan hệ ngữ nghĩa giữa chúng.
- **Hệ thống Nhãn Miền Phân cấp (Hierarchical Domain Tag System):** $\mathcal{G}_d = (\mathcal{V}_d, \mathcal{E}_d)$, trong đó $\mathcal{V}_d$ chứa các nhãn miền ở nhiều mức độ trừu tượng khác nhau và $\mathcal{E}_d$ biểu diễn quan hệ phân cấp cha – con (*"has subdomain"*) xuất phát từ gốc $\hat{v}$.
- **Đồ thị Tích hợp Toàn cục:** $\mathcal{G} = (\mathcal{V}_o, \mathcal{E}_o, \mathcal{V}_d, \mathcal{E}_d, \mathcal{E}_{od})$, với $\mathcal{E}_{od}$ là tập các cạnh liên kết giữa nhãn thực thể $\mathcal{V}_o$ và nhãn miền $\mathcal{V}_d$. Khi nhận câu hỏi q, hệ thống sinh câu trả lời toàn cục $a = \mathcal{F}(q, \mathcal{G})$.

Giai đoạn & Bước xử lý	Công thức Toán học & Cơ chế Thực thi	Vai trò & Ưu thế Kỹ thuật
Giai đoạn 1.1: Trích xuất từ khóa Nhãn Thực thể (*Object Tag Extraction*)	Chia D thành các đoạn văn bản $T = \{t_i\}_{i=1}^{\|T\|}$ (kích thước 1200 tokens, gối nhau 100 tokens). Gọi LLM trích xuất từ khóa kèm mô tả: $\mathcal{V}_o, \mathcal{E}_o = LLM(\{t_i\}_{i=1}^{\|T\|})$	Thu thập tri thức nền tảng chi tiết nhất (primitive grounding knowledge) từ văn bản gốc mà không bị nhiễu bởi các thực thể rác.
Giai đoạn 1.2: Tổ chức Chuỗi Nhãn Miền (*Domain Tag Chain Organization*)	Từ mỗi $v_o \in \mathcal{V}_o$ và miền gốc $\hat{v}$, LLM sinh chuỗi phân cấp đa tầng: $\mathcal{C} = \{c_i\}_{i=1}^{\|\mathcal{V}_o\|} = LLM(\mathcal{V}_o, \hat{v})$ Hợp nhất tất cả chuỗi $c_i \in \mathcal{C}$ vào cấu trúc **DAG** theo Thuật toán 1.	Loại bỏ hoàn toàn phụ thuộc vòng (cyclic dependencies) và liên kết dư thừa; tạo trục phân cấp rõ ràng từ tổng quát đến chi tiết.

Giai đoạn & Bước xử lý	Công thức Toán học & Cơ chế Thực thi	Vai trò & Ưu thế Kỹ thuật
Giai đoạn 1.3: Hợp nhất Tri thức Tâm Miền (*Domain-centric Knowledge Fusion*)	Với mỗi nhãn miền $v_d \in \mathcal{V}_d$, tổng hợp chuỗi $Chain(v_d)$ và các nhãn thực thể lân cận $Nei(v_d)$ thành bản tóm tắt: $s = LLM(Chain(v_d), Nei(v_d))$ Vector hóa vào thư viện: $\mathbb{K} \leftarrow \{v_i, s_i, Emb(s_i)\}_{i=1}^{\lvert \mathcal{V}_d \rvert}$	Đóng gói sẵn cả tầm nhìn vĩ mô của chuỗi miền lẫn chi tiết chuyên môn của thực thể vào từng nút miền trước khi truy vấn.
Giai đoạn 1.4: Chèn Tri thức Tăng trưởng (*Incremental Insertion*)	Khi có tài liệu mới: (1) *Tag increment:* nối thêm mô tả vào nhãn trùng tên hoặc gắn nhánh mới vào DAG; (2) *Knowledge increment:* chỉ tóm tắt lại bản tóm tắt cũ và mới tại các nút miền bị ảnh hưởng.	Tránh việc phải phân cụm lại toàn bộ đồ thị như thuật toán Leiden của GraphRAG; mở rộng quy mô tuyến tính.
Giai đoạn 2.1: Truy xuất Tri thức Tâm Miền (*Domain-centric Retrieval*)	Với câu hỏi q, tìm top-k ($k = 3$) nhãn miền gần nhất qua độ tương đồng Cosine trên thư viện vector $\mathbb{K}$: $(\mathcal{V}'_t, S'_t) = Ret_{tag}(q, \mathbb{K})$	Định vị chính xác vùng tri thức liên quan chỉ bằng tìm kiếm vector đơn giản, không cần duyệt toàn bộ đồ thị.
Giai đoạn 2.2 & 2.3: Tích hợp Chuỗi Phân cấp & Sinh câu trả lời (*Chain Integration & Generation*)	Truy xuất chuỗi phân cấp đi kèm $(\mathcal{V}'_c, S'_c) = Ret_{chain}(\mathcal{V}'_t, \mathcal{G})$. Ưu tiên đưa S'_t rồi đến S'_c vào ngữ cảnh LLM: $a = LLM(q, S'_t, S'_c)$	Cung cấp đồng thời tri thức đặc thù tại nút miền và ngữ cảnh thượng tầng dọc theo chuỗi DAG để sinh câu trả lời toàn diện.

Giải thích chi tiết Thuật toán 1 (Domain Tag Chain Organization thành cấu trúc DAG)

Trong mục 4.1.2, sau khi LLM sinh ra danh sách các chuỗi miền $\mathcal{C}$ (ví dụ: `Computer Science → Data Processing → Big Data → Apache Spark`), nếu chỉ lưu rời rạc thì các nhánh chung sẽ bị trùng lặp. Thuật toán 1 khởi tạo một đồ thị có hướng không chu trình `DAG = {}` rỗng. Với mỗi chuỗi $c_i \in \mathcal{C}$, thuật toán duyệt chỉ số j từ 2 đến $|c_i| - 1$: tìm nút cha $p_{i,j} = get_node(DAG, c_{i,j-1})$ và nút con hiện tại $n_{i,j} = get_node(DAG, c_{i,j})$. Nếu nút con $n_{i,j}$ đã tồn tại trong DAG nhưng chưa nằm trong danh sách con của $p_{i,j}$, thuật toán nối cạnh trực tiếp `p_{i,j}.add_child(n_{i,j})`; nếu chưa tồn tại, thuật toán khởi tạo nút mới `n_new = new_node(c_{i,j})` rồi gắn vào $p_{i,j}$. Cuối cùng, hàm `merge_graph(`$\mathcal{G}$`, DAG)` hợp nhất cây phân cấp vào đồ thị tổng thể, đảm bảo tính bắc cầu (transitivity) và duy trì độ tương đồng ngữ nghĩa giữa hai tầng kề nhau đạt trung bình **0.80 – 0.85** (Bảng 11 của bài báo).

3. Hệ thống Prompt Chuẩn trong Bài báo (Appendix C)

Một lợi thế rất lớn khi chọn đề tài **TagRAG** làm đồ án môn NLP là nhóm tác giả đã công bố toàn bộ 6 mẫu Prompt trong Phụ lục C (trang 6454–6456). Khi cài đặt hệ thống, sinh viên cần nắm vững 3 mẫu Prompt xây dựng đồ thị quan trọng nhất dưới đây:

[PROMPT 1: Object Tag Keyword Extraction (Mục 4.1.1)] -Goal- Given a domain-specific text document and a list of keyword types, summarize keywords from the text and generate their relationships. Use {language} as output language. -Steps- 1. Summarize keywords from the text: generate keyword_name (capitalized if English), keyword_type, and keyword_description (comprehensive description of attributes and activities). Format: ("keyword"{tuple_delimiter}<keyword_name>{tuple_delimiter}<keyword_type>{tuple_delimiter}<keyword_description>) 2. Generate all pairs of (source_keyword, target_keyword) that are *clearly related* to each other with relationship_description. Format: ("relationship"{tuple_delimiter}<source_keyword>{tuple_delimiter}<target_keyword>{tuple_delimiter}<relationship_description>) **[PROMPT 2: Domain Tag Chain Organization (Mục 4.1.2)]** -Goal- Given a root domain tag with its description and an object tag with its description, generate the relationship chain between them. -Steps- 1. Present all domain tags in the chain: Format each as <domain_name>{explanation_delimiter}<domain_description> and connect domain tags with **{inference_delimiter}**. 2. Generate the relationship description between the object tag and the generated relationship chain using **{tuple_delimiter}**. **[PROMPT 3: Domain-centric Knowledge Fusion (Mục 4.1.3)]** -Goal- Given a chain of domain tags with their descriptions ({domain_tag_chain}), the summary of the chain ({chain_summary}), relevant object tags ({object_tags}) and their relationships ({relationships}), summarize the domain by injecting these object tags at a high level.

4. Thiết lập Thực nghiệm & Tổng hợp Kết quả Đánh giá

Nhóm tác giả đánh giá TagRAG trên 4 tập dữ liệu thuộc bộ chuẩn **UltraDomain** (**Qian et al., 2025**) gồm: **Agriculture** (12 tài liệu, 1.756 chunks, 2,01 triệu tokens), **Computer Science – CS** (10 tài liệu, 1.858 chunks, 2,30 triệu tokens), **Legal** (94 tài liệu, 4.294 chunks, 5,08 triệu tokens) và **Mix** (61 tài liệu đa ngành từ 18 lĩnh vực, 579 chunks, 619 nghìn tokens). Trên mỗi tập, `GPT-4o-mini` sinh 125 câu hỏi toàn cục (5 nhóm người dùng × 5 tác vụ × 5 câu hỏi).

Cấu hình mô hình & Cơ chế chấm điểm LLM-as-a-Judge: Mô hình xương sống (backbone) mặc định là `Qwen3-4B` (**Qwen Team, 2025**), mô hình nhúng là `bge-large-en-v1.5` (**Xiao et al., 2023**) lưu trên cơ sở dữ liệu vector nhẹ `nano-vectordb` với $top\text{-}k = 3$. Để đảm bảo tính khách quan tuyệt đối, mỗi cặp câu trả lời được chấm đối kháng (head-to-head winning rate) trên 4 tiêu chí: **Comprehensiveness** (Độ toàn diện), **Diversity** (Độ đa dạng góc nhìn), **Empowerment** (Khả năng hỗ trợ ra quyết định) và **Overall** (Tổng thể) bởi **3 giám khảo độc lập** (`GPT-4o-mini`, `gemini-2.5-pro`, `claude-sonnet-4.5`) kết hợp đảo thứ tự câu trả lời (3 × 2 = 6 lượt chấm/câu hỏi, đạt tỷ lệ đồng thuận đa số 93.9%).

Đối thủ So sánh (Baselines)	Tiêu chí	Agri (%)	CS (%)	Legal (%)	Mix (%)	Trung bình (%)
v.s. NaiveRAG (Lewis et al., 2020)	Comprehensiveness / Overall	93.9 / **93.5**	94.0 / **89.7**	79.5 / **72.3**	90.0 / **87.9**	89.3 / **85.8**
v.s. GraphRAG (Edge et al., 2024)	Comprehensiveness / Overall	89.3 / **90.9**	85.3 / **81.9**	60.7 / **62.3**	67.1 / **67.7**	75.6 / **75.7**
v.s. LightRAG (Guo et al., 2025)	Comprehensiveness / Overall	94.4 / **94.4**	85.1 / **84.0**	80.3 / **79.9**	88.1 / **89.7**	87.0 / **87.0**
v.s. MiniRAG (Fan et al., 2025)	Comprehensiveness / Overall	76.0 / **78.3**	55.2 / **57.5**	56.8 / **58.4**	64.1 / **65.3**	63.0 / **64.9**
v.s. Zero-shot Qwen3-30B-A3B (Lớn gấp 7.5×)	Comprehensiveness / Overall	84.8 / **80.3**	72.7 / **55.5**	68.8 / **51.6**	59.6 / **44.7**	71.5 / **58.0**
Ablation: TagRAG v.s. w/o Chain	Comprehensiveness / Overall	87.1 / **87.2**	90.5 / **87.5**	85.2 / **80.1**	78.9 / **75.2**	85.4 / **82.5**
Ablation: TagRAG v.s. w/o Fusion	Comprehensiveness / Overall	97.3 / **96.9**	95.5 / **89.7**	95.5 / **88.0**	85.9 / **78.4**	93.6 / **88.3**

Năm Phát hiện Thực nghiệm Đắt giá Cần Nhấn mạnh Khi Bảo vệ Đồ án

1. **Vượt trội cả về chất lượng lẫn tốc độ (14.6× Construction & 1.9× Retrieval):** So với GraphRAG, TagRAG rút ngắn thời gian dựng đồ thị tới 14.6 lần và thời gian suy luận 1.9 lần, đồng thời thắng GraphRAG **75.7%** và thắng LightRAG **87.0%**. LightRAG tuy dựng đồ thị nhanh nhất nhưng chỉ trích xuất được 2.787 cạnh quan hệ trên tập CS (so với 32.258 cạnh của TagRAG), gây đứt gãy ngữ nghĩa nghiêm trọng.
2. **Khả năng thích nghi xuất sắc với Mô hình Ngôn ngữ Cực nhỏ (1.5B – 1.7B):** Khi hạ mô hình xương sống xuống `glm-edge-1.5b-chat` (Bảng 8), GraphRAG, LightRAG và MiniRAG sụp đổ hoàn toàn; TagRAG đạt tỷ lệ thắng tuyệt đối **93.1% trước GraphRAG**, **94.6% trước LightRAG** và **93.3% trước MiniRAG**. Thậm chí TagRAG chạy trên `Qwen3-1.7B` vẫn thắng GraphRAG chạy trên `Qwen3-4B` tới **67.7%** (Bảng 9).
3. **Cập nhật tăng trưởng đa vòng & xuyên miền (Incremental & Cross-domain Upsert):** Qua 10 vòng chèn tài liệu liên tiếp trên tập CS (Hình 4), TagRAG duy trì tỷ lệ thắng ổn định trên 80%. Khi chèn tài liệu CS vào đồ thị Mix (Bảng 4), câu trả lời sau cập nhật của TagRAG thắng trước cập nhật **58.0%** (chỉ mất 6.37 giờ cập nhật và 2.47 giờ suy luận), trong khi GraphRAG tụt xuống **44.0%** (bị suy giảm chất lượng do cấu trúc cộng đồng cũ cản trở miền mới) và tiêu tốn tới 30.47 giờ cập nhật.
4. **Vai trò sống còn của Chuỗi Nhãn (Chain) và Hợp nhất Tri thức (Fusion):** Trong nghiên cứu cắt bỏ (Ablation Study – Bảng 2), nếu bỏ bước hợp nhất tri thức (`w/o fusion`, chỉ dùng mô tả nhãn miền trần), chất lượng sụt giảm thê thảm (TagRAG đầy đủ thắng **88.3%** trung bình và lên tới **96.9%** trên tập Agri). Nếu bỏ bước tích hợp chuỗi phân cấp lúc truy xuất (`w/o chain`), TagRAG đầy đủ thắng **82.5%**.

5. **Không phụ thuộc vào bộ truy xuất vector lớn (Retriever Robustness):** Khi thay `bge-large-en-v1.5` bằng các mô hình nhúng nhỏ hơn là `bge-base` và `bge-small` (Bảng 3), tỷ lệ thắng của TagRAG trước GraphRAG và MiniRAG thậm chí còn tăng nhẹ (từ 58.4% lên 62.8% trước MiniRAG), chứng minh cấu trúc chuỗi DAG đã bù đắp hoàn hảo cho sai số của bộ tìm kiếm vector.

5. Hướng dẫn Triển khai Đồ án Cuối kỳ NLP (30% Điểm)

Vì TagRAG là bài báo thuần văn bản (text-only) và được thiết kế chuyên biệt cho kịch bản ít tài nguyên tính toán, nhóm của bạn có lợi thế rất lớn để hoàn thành một đồ án xuất sắc hoặc nâng cấp thành bài báo khoa học. Dưới đây là 3 phương án triển khai thực tế:

Phương án Đề tài	Mục tiêu & Phạm vi Thực nghiệm	Điểm Đóng góp & Công cụ Khuyên dùng
Phương án 1 (Cơ bản – Mục tiêu 8.5/10): Tái hiện TagRAG & Khảo sát Small LLMs	Xây dựng lại toàn bộ pipeline 2 giai đoạn của TagRAG trên một phần bộ dữ liệu **UltraDomain (CS hoặc Mix)** (khoảng 100–200 chunks để chạy nhanh trên Colab). So sánh trực tiếp TagRAG với *NaiveRAG*, *LightRAG* và *TagRAG w/o Chain*.	Chứng minh hiệu quả của cấu trúc chuỗi DAG trên các mô hình mở nhẹ như `Qwen2.5-3B/7B`, `Llama-3.2-3B` hoặc `Gemini-2.5-Flash` cùng `nano-vectordb` và `bge-small-en-v1.5`.
Phương án 2 (Khá/Giỏi – Mục tiêu 9.0–9.5/10): Xây dựng *VietTagRAG* cho Văn bản Pháp luật / Y khoa / Giáo dục Việt Nam	Áp dụng kiến trúc TagRAG vào miền dữ liệu tiếng Việt có tính phân cấp mạnh: ví dụ **Bộ luật Dân sự / Luật Giao thông / Quy chế Học vụ Đại học** hoặc **Cẩm nang Chẩn đoán Y khoa Bộ Y tế** (tạo bộ 50 câu hỏi tổng hợp toàn cục).	Thiết kế bộ Nhãn Miền Gốc tiếng Việt ($\upsilon\hat{}$ = *"Pháp luật Việt Nam"*), dùng mô hình nhúng đa ngôn ngữ `BAAI/bge-m3` hoặc `keepitreal/vietnamese-sbert`. Trình diễn tính năng **Chèn văn bản luật sửa đổi bổ sung (Incremental Insertion)**.
Phương án 3 (Hướng Nghiên cứu / Viết Paper – Mục tiêu 9.5–10/10): Cải tiến Hạn chế Chi phí LLM của TagRAG (*Hybrid Ontology/Clustering TagRAG*)	Giải quyết trực tiếp hạn chế được tác giả nêu ở mục *Limitations* (trang 6442): TagRAG hiện phải gọi LLM cho từng Object Tag để sinh Domain Tag Chain, gây tốn kém và đôi khi sinh nhãn miền không đồng nhất (Jaccard chuỗi đạt 0.7396).	Đề xuất **Gom cụm Ngữ nghĩa Trước (Pre-clustering Object Tags)** hoặc dùng **Khung Phân loại Học thuật (Taxonomy/Ontology-guided Constraint)** để gom nhóm các Object Tags gần nhau rồi mới gọi LLM sinh chuỗi miền theo lô (Batch Chain Generation), giúp giảm 60–80% số lần gọi LLM ở bước 4.1.2!

Quy trình 5 Bước Cài đặt Code Python cho Nhóm (Implementation Roadmap)

1. **Bước 1 – Chuẩn bị Dữ liệu & Chia đoạn (Chunking):** Dùng `RecursiveCharacterTextSplitter` hoặc cắt theo token (`tiktoken` / `AutoTokenizer`) với `chunk_size = 1200`, `chunk_overlap = 100`. Định nghĩa từ điển nhãn miền gốc `ROOT_DOMAIN = {"name": "...", "description": "..."}`.
2. **Bước 2 – Module Trích xuất Nhãn Thực thể ($\mathscr{V}_o$, $\mathscr{E}_o$):** Cài đặt Prompt 1 gọi LLM (khuyến nghị dùng `vLLM`, `Ollama` hoặc API bất đồng bộ `asyncio`), phân tách kết quả trả về theo dấu phân cách `{tuple_delimiter}` và hợp nhất mô tả của các từ khóa trùng tên.
3. **Bước 3 – Module Tổ chức Chuỗi Miền & Dựng DAG (Algorithm 1):** Gọi Prompt 2 để sinh đường dẫn từ `ROOT_DOMAIN` tới từng $v_o \in \mathscr{V}_o$. Dùng thư viện `networkx.DiGraph()` trong Python để cài đặt chính xác 16 dòng mã giả của **Algorithm 1**, kiểm tra tính chất không chu trình bằng `nx.is_directed_acyclic_graph(G)`.
4. **Bước 4 – Module Hợp nhất Tri thức & Lưu trữ Vector ($\mathbb{K}$):** Duyệt qua các nút miền $v_d \in \mathscr{V}_d$ trong đồ thị `networkx`, trích xuất đường đi từ gốc tới v_d làm `Chain(v_d)` và các nút lá v_o kề nó làm `Nei(v_d)`. Gọi Prompt 3 sinh bản tóm tắt s_i, mã hóa bằng `SentenceTransformer` và lưu vào `nano-vectordb` hoặc `FAISS`.
5. **Bước 5 – Module Truy xuất Hướng dẫn bởi Nhãn & Đánh giá Tự động:** Khi nhận truy vấn q, lấy top-3 nút miền từ Vector DB (S'_t), dùng hàm `nx.ancestors()` / đường dẫn chuỗi trên DAG để lấy thêm bản tóm tắt của các miền cha (S'_c), ghép vào Prompt 4 để sinh câu trả lời. Cuối cùng, chạy script chấm điểm tự động đối kháng theo Prompt Evaluation ở trang 6454.

6. Bố cục Slide Seminar (10%) & Bộ Câu hỏi Vấn đáp Trọng tâm (10%)

Để tối ưu hóa điểm số ở buổi thuyết trình **Seminar (10%)** và **Vấn đáp trực tiếp (10%)**, nhóm cần phân bổ bài trình bày (12–15 phút) thành 5 phần rõ ràng như sau:

Phần Trình bày	Số Slide	Nội dung Cốt lõi Cần Nhấn mạnh trên Slide
1. Đặt vấn đề & Hạn chế của GraphRAG	2–3 slides	So sánh trực quan NaiveRAG (thiếu tầm nhìn toàn cục) và GraphRAG (phân cụm Leiden tốn kém, phụ thuộc LLM lớn, không thể cập nhật thêm tài liệu mà không dựng lại đồ thị); nêu 2 câu hỏi nghiên cứu RQ1 & RQ2.
2. Kiến trúc TagRAG & Thuật toán DAG	3–4 slides	Trình bày sơ đồ tổng thể (Hình 2 bài báo): phân biệt *Object Tags* ($\mathscr{V}_o$) và *Domain Tag Chains* ($\mathscr{V}_d$). Giải thích Thuật toán 1 hợp nhất chuỗi thành DAG và công thức hợp nhất tri thức $s = LLM(Chain(v_d), Nei(v_d))$.
3. Cơ chế Cập nhật Tăng trưởng & Truy xuất	2 slides	Giải thích tại sao TagRAG chèn tài liệu mới không cần dựng lại đồ thị (*Tag increment* + *Knowledge increment*) và cách kết hợp S'_t với S'_c lúc suy luận.
4. Kết quả Bài báo & Thực nghiệm của Nhóm	3–4 slides	Biểu đồ bong bóng Hiệu năng – Thời gian (14.6× nhanh hơn GraphRAG), kết quả trên mô hình siêu nhỏ 1.5B/1.7B, kết quả Ablation (`w/o chain`, `w/o fusion`) và **Demo kết quả thực nghiệm của nhóm**.

Phần Trình bày	Số Slide	Nội dung Cốt lõi Cần Nhấn mạnh trên Slide
5. Thảo luận Hạn chế & Hướng Cải tiến	1–2 slides	Chi phí gọi LLM khi sinh chuỗi cho từng Object Tag, giới hạn dữ liệu đa phương thức (ảnh/bảng biểu) và đề xuất cải tiến gom cụm trước của nhóm.

Bộ 4 Câu hỏi & Đáp án Chuẩn bị cho Vấn đáp (Q&A Defense)

- **Câu hỏi 1: Sự khác biệt bản chất nhất giữa cách xây dựng phân cấp của TagRAG và GraphRAG là gì?**
  Trả lời: GraphRAG xây dựng hoàn toàn từ dưới lên (bottom-up) bằng cách dùng thuật toán phát hiện cộng đồng (như Leiden) dựa trên mật độ cạnh giữa các thực thể, nên các cộng đồng không có định hướng chủ đề rõ ràng và phải tính toán lại toàn bộ khi thêm nút mới. Ngược lại, TagRAG kết hợp hướng từ dưới lên (trích xuất Object Tags $\mathcal{V}_o$) với định hướng từ trên xuống thông qua một Nhãn Miền Gốc định trước ($\hat{v}$) để tạo thành các chuỗi phân cấp logic (Domain Tag Chains) rồi hợp nhất thành cấu trúc DAG. Nhờ bộ khung DAG cố định theo miền, tài liệu mới chỉ cần gắn nhánh vào đúng vị trí mà không phá vỡ cấu trúc cũ.
- **Câu hỏi 2: Tại sao trong thí nghiệm với mô hình cực nhỏ (glm-edge-1.5b-chat và Qwen3-1.7B), GraphRAG và MiniRAG lại suy sụp trong khi TagRAG vẫn hoạt động tốt?**
  Trả lời: GraphRAG đòi hỏi LLM phải tổng hợp hàng chục thực thể rời rạc trong một cộng đồng lớn, còn MiniRAG/PathRAG đòi hỏi LLM phải tự suy luận bắc cầu qua các đường đi đồ thị phức tạp lúc truy vấn — những tác vụ vượt quá khả năng của mô hình 1.5B. Trong khi đó, TagRAG đã chia nhỏ việc tổ chức tri thức thành các chuỗi phân cấp ngắn và thực hiện bước *Domain-centric Knowledge Fusion* từ trước, giúp đóng gói sẵn tri thức toàn cục vào từng bản tóm tắt miền s_i. Lúc suy luận, mô hình 1.5B chỉ cần đọc các bản tóm tắt đã được chắt lọc sẵn trong S'_t và S'_c.
- **Câu hỏi 3: Nếu bộ dữ liệu bao gồm nhiều lĩnh vực hoàn toàn khác nhau (như tập UltraDomain Mix gồm 18 ngành), làm sao chọn được Nhãn Miền Gốc $\hat{v}$?**
  Trả lời: Theo Bảng 7 và mục B.8 (Hình 6) của bài báo, đối với tập dữ liệu đa ngành, tác giả đặt nhãn miền gốc khái quát là `"ALL DISCIPLINES"`. Từ gốc này, LLM tự động phân nhánh xuống các đại lĩnh vực ở tầng 1 như `HUMANITIES`, `SOCIAL SCIENCES`, `HISTORICAL STUDIES`, `TECHNOLOGY` rồi mới đi sâu xuống các chuyên ngành hẹp (ví dụ thực thể `"BERT"` được nối vào nhánh `NATURAL LANGUAGE PROCESSING`, còn `"THIRTY YEARS' WAR"` nối vào `HISTORICAL STUDIES → EUROPEAN HISTORY`).
- **Câu hỏi 4: Điểm yếu lớn nhất của TagRAG nằm ở bước nào trong pipeline và nhóm em đề xuất khắc phục ra sao?**
  Trả lời: Điểm nghẽn nằm ở Công thức $\mathcal{C} = \{c_i\}_{i=1}^{|\mathcal{V}_o|} = LLM(\mathcal{V}_o, \hat{v})$: số lượng Object Tags $|\mathcal{V}_o|$ thường rất lớn (ví dụ 14.768 nhãn trên tập CS), việc gọi LLM để sinh chuỗi cho từng nhãn thực thể vẫn tốn nhiều lệnh gọi và có thể sinh ra các tên miền trung gian đồng nghĩa nhưng khác cách viết (Jaccard chuỗi giữa 2 lần chạy là 0.7396). Nhóm có thể khắc phục bằng cách nhúng vector các Object Tags, chạy phân cụm K-Means/HDBSCAN để gom các Object Tags cùng chủ đề lại rồi chỉ gọi LLM sinh một chuỗi miền chung cho cả cụm.

7. Tài liệu Tham khảo Cốt lõi (References)

1. **Tao, W., Li, X., Lan, Y., & Qian, W. (2026).** TagRAG: Tag-guided Hierarchical Knowledge Graph Retrieval-Augmented Generation. *Findings of the Association for Computational Linguistics: ACL 2026*, pages 6434–6456.
2. **Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., & Larson, J. (2024).** From Local to Global: A Graph RAG Approach to Query-Focused Summarization (GraphRAG). *arXiv preprint arXiv:2404.16130*.
3. **Guo, Z., Xia, L., Yu, Y., Ao, T., & Huang, C. (2025).** LightRAG: Simple and Fast Retrieval-Augmented Generation. *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 10746–10761.
4. **Fan, T., Wang, J., Ren, X., & Huang, C. (2025).** MiniRAG: Towards Extremely Simple Retrieval-Augmented Generation. *arXiv preprint arXiv:2501.06713*.
5. **Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020).** Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9474.
6. **Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., & Manning, C. D. (2024).** RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. *The Twelfth International Conference on Learning Representations (ICLR 2024)*.
7. **Huang, H., Huang, Y., Yang, J., Pan, Z., Chen, Y., Ma, K., Chen, H., & Cheng, J. (2025a).** Retrieval-Augmented Generation with Hierarchical Knowledge (HiRAG). *arXiv preprint arXiv:2503.10150*.
8. **Huang, Y., Zhang, S., & Xiao, X. (2025b).** KET-RAG: A Cost-Efficient Multi-Granular Indexing Framework for Graph-RAG. *arXiv preprint arXiv:2502.09304*.
9. **Hong, Y., Li, C., Zhang, J., & Shao, Y. (2025).** FG-RAG: Enhancing Query-Focused Summarization with Context-Aware Fine-Grained Graph RAG. *arXiv preprint arXiv:2504.07103*.
10. **Chen, B., Guo, Z., Yang, Z., Chen, Y., Chen, J., Liu, Z., Shi, C., & Yang, C. (2025a).** PathRAG: Pruning Graph-Based Retrieval Augmented Generation with Relational Paths. *arXiv preprint arXiv:2502.14902*.
11. **Zhang, Y., Wu, R., Cai, P., Wang, X., Yan, G., Mao, S., Wang, D., & Shi, B. (2026).** LeanRAG: Knowledge-Graph-Based Generation with Semantic Aggregation and Hierarchical Retrieval. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40, 34862–34869.
12. **Qian, H., Liu, Z., Zhang, P., Mao, K., Lian, D., Dou, Z., & Huang, T. (2025).** MemoRAG: Boosting Long Context Processing with Global Memory-Enhanced Retrieval Augmentation (UltraDomain Benchmark). *Proceedings of the ACM on Web Conference 2025*, pages 2366–2377.
13. **Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2023).** C-Pack: Packaged Resources to Advance General Chinese Embedding (BGE). *arXiv preprint arXiv:2309.07597*.
14. **Qwen Team. (2025).** Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*.